

MicroCRM — Orion

Documentation CI/CD complete

Partie 1 — Mise en œuvre CI/CD, conteneurisation, testing
Partie 2 — KPI, métriques, sécurité, sauvegarde, mises à jour

Auteur : Hassane CHKIRROU — Septembre 2026

PARTIE 1

1. Étapes de mise en œuvre CI/CD

Cette section détaille, pour chaque commande importante du pipeline, son objectif, l'endroit où elle est définie et le moment où elle s'exécute.

1.1 Job « backend »

Commande	Objectif	Où elle est définie	Quand
<code>./gradlew build</code>	Compiler le code et exécuter les tests unitaires JUnit	<code>.github/workflows/ci.yml</code> (job backend) / localement	À chaque push sur dev ou main, et en local avant commit
<code>./gradlew sonar</code>	Envoyer le rapport d'analyse à SonarQube Cloud (bugs, vulnérabilités, couverture)	<code>ci.yml</code> , après le build	À chaque exécution du job backend

1.2 Job « frontend »

Commande	Objectif	Où elle est définie	Quand
<code>npm ci</code>	Installer les dépendances de façon reproductible (à partir de <code>package-lock.json</code>)	<code>ci.yml</code> (job frontend) / <code>package.json</code>	À chaque exécution du job frontend
<code>npx ng test --code-coverage</code>	Exécuter les tests unitaires Karma/Jasmine et mesurer la couverture	<code>ci.yml</code>	À chaque push / PR
<code>npx ng build --configuration production</code>	Compiler le bundle Angular optimisé pour la production	<code>ci.yml</code>	Après les tests, à chaque exécution
<code>sonarqube-scan-action</code>	Envoyer l'analyse de code TypeScript/CSS à SonarQube Cloud	<code>ci.yml</code> , via <code>sonar-project.properties</code>	À chaque exécution du job frontend

1.3 Job « publish-images » (déploiement continu)

Commande	Objectif	Où elle est définie	Quand
<code>docker/login-action</code>	S'authentifier auprès de GitHub Container Registry (GHCR)	<code>ci.yml</code> (job publish-images)	Uniquement sur push vers main, après succès de backend et frontend
<code>docker/build-push-action</code> (back)	Construire l'image du stage back du Dockerfile et la publier sur GHCR	<code>ci.yml</code>	Idem
<code>docker/build-push-action</code> (front)	Idem pour le stage front	<code>ci.yml</code>	Idem

Le job `publish-images` dépend explicitement (*needs*) des jobs `backend` et `frontend` : il ne s'exécute que si les deux ont réussi, et uniquement sur la branche `main` (condition `if: github.ref == 'refs/heads/main'`).

1.4 Workflow « release »

Commande	Objectif	Où elle est définie	Quand
<code>./gradlew build -x test</code>	Construire le JAR exécutable (tests déjà validés par le pipeline CI)	release.yml	À la création d'un tag vX.Y.Z
<code>npx ng build --configuration production</code>	Construire le bundle de production Angular	release.yml	Idem
<code>zip -r microcrm-front.zip</code>	Empaqueter le build Angular en une archive téléchargeable	release.yml	Idem
<code>softprops/action-gh-release</code>	Créer la release GitHub, générer le changelog automatiquement, attacher les artefacts	release.yml	Idem, dernière étape du workflow

1.5 Historique réel des exécutions CI/CD

Capture de l'onglet Actions du dépôt GitHub, illustrant l'historique réel des workflows exécutés au fil du projet (succès et échecs, dont ceux analysés en section 4.1 pour le calcul du Change Failure Rate) :

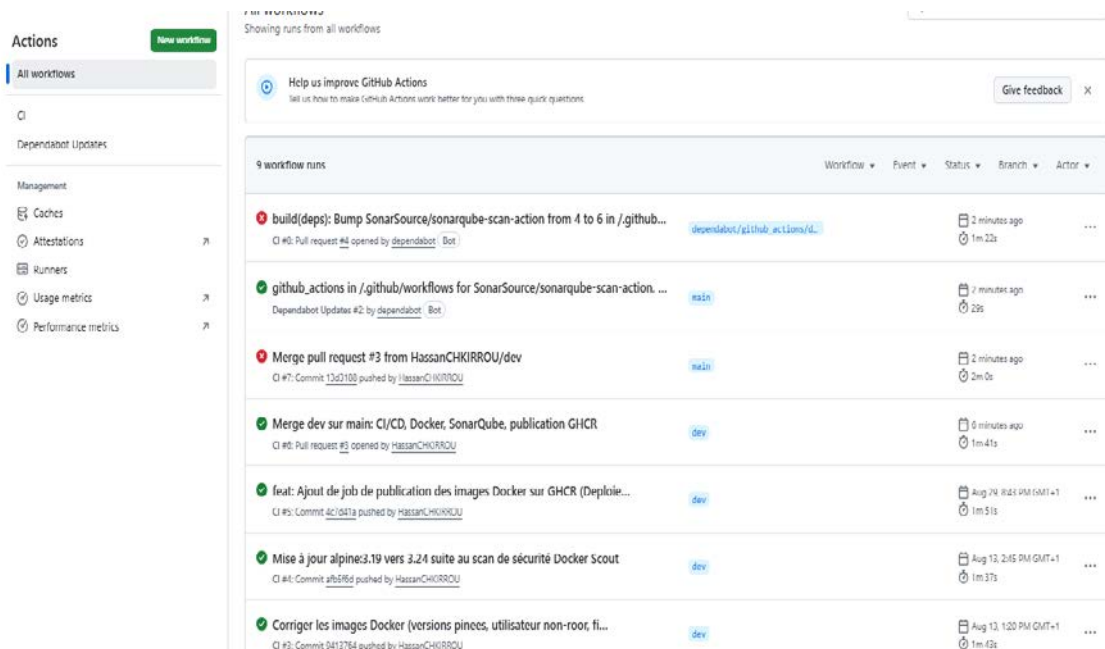


Figure 1 — Historique des workflows GitHub Actions (CI et Dependabot Update)

2. Plan de conteneurisation et de déploiement

2.1 Comment l'application est déployée

- 1. Développement sur la branche dev, tests locaux (./gradlew build, npm test)
- 2. Push sur dev → le pipeline CI construit et teste le back-end et le front-end, puis lance l'analyse SonarQube
- 3. Pull Request de dev vers main → validation humaine volontaire avant mise en production
- 4. Merge sur main → publication automatique des images Docker sur GHCR (taguées latest et par SHA de commit)
- 5. (Optionnel) Création d'un tag vX.Y.Z → génération d'une release GitHub avec artefacts et changelog

2.2 Prérequis techniques

Environnement	Prérequis
Développement local	Java 17+, Node.js 20+/npm, Docker Desktop, Git
CI/CD (GitHub Actions)	Secret SONAR_TOKEN configuré ; GITHUB_TOKEN natif pour GHCR
Exécution locale (Docker Compose)	Ports 80, 443 et 8080 disponibles sur la machine hôte

2.3 Images Docker retenues

Stage	Image	Justification
front-build	node:20-alpine	Image officielle, version pinnée, variante Alpine minimale
back-build	gradle:8.7-jdk17	Version exacte pinnée pour un build reproductible
front (final)	alpine:3.24	Distribution minimale officielle, mise à jour suite à l'audit Docker Scout
back (final)	eclipse-temurin:21-jre-alpine	Image officielle Eclipse Adoptium, JRE seul (pas de JDK)

Le passage de alpine:3.19 à alpine:3.24 (audit Docker Scout) a réduit les vulnérabilités de l'image front-end de 17 critiques / 36 hautes / 57 moyennes / 9 faibles à 0 / 2 / 4 / 0.

2.4 Ordre de démarrage (Docker Compose)

```
docker compose up --build
```

Le service back-end démarre en premier ; un health check interroge /persons jusqu'à ce qu'il réponde. Le service front-end (depends_on: condition: service_healthy) ne démarre qu'une fois le back-end opérationnel.

Justification : ce flux dev → CI → PR → main → publication est directement motivé par l'observation faite en partie 2 (Change Failure Rate de 75 % lors des premiers essais de publication) — la PR intermédiaire agit comme filet de sécurité supplémentaire.

3. Plan de testing périodique

3.1 Types de tests exécutés

Côté	Type	Outil	Couvre
Back-end	Unitaires + intégration	JUnit 5 (Gradle)	Contexte Spring, accès aux données (HSQLDB)
Front-end	Unitaires	Karma + Jasmine (headless Chrome)	Composants et services Angular
Back-end / Front-end	Analyse statique	SonarQube Cloud	Bugs, code smells, vulnérabilités, couverture
Full-stack	Validation d'intégration	Build Docker multi-stage	L'application démarre réellement une fois empaquetée

3.2 Déclencheurs

Événement	Jobs exécutés
Push sur une branche de feature / dev	Tests unitaires front + back (retour rapide)
Pull Request vers main	Tests unitaires + analyse SonarQube + build Docker (contrôle avant fusion)
Merge sur main	Tests + Sonar + build Docker + publication de l'image (GHCR)
Création d'un tag vX.Y.Z	Build des artefacts + création de la release GitHub

3.3 Objectifs associés

- Validation fonctionnelle : garantir que chaque fonctionnalité (gestion des Person/Organization) se comporte comme attendu
- Non-régression : exécution systématique sur chaque Pull Request avant toute fusion
- Qualité de code continue : suivi de la dette technique et de la couverture dans la durée via SonarQube
- Fiabilité du déploiement : le build Docker en CI valide que ce qui sera déployé démarre réellement

PARTIE 2

4. KPI proposés et métriques

4.1 Métriques DORA

Calculées à partir des données réelles de l'historique GitHub Actions du projet (période du 11 au 31 août 2026).

Métrique	Valeur mesurée	Méthode de calcul
Deployment Frequency	~1 publication d'image / période ; 2 releases sur 3 semaines	Comptage des exécutions réussies de publish-images et des tags créés
Lead Time for Changes	~1m 42s en moyenne (pipeline CI)	Durée entre le commit poussé et la fin du build/test, sur 4 exécutions
Change Failure Rate	75 % (3 échecs sur 4 tentatives)	Tentatives du déploiement publish-images jusqu'au premier succès
Mean Time to Restore	~15 minutes en moyenne	Temps entre la détection d'un échec CI et le commit correctif

Détail — Lead Time (4 exécutions mesurées)

Run	Date / heure	Durée du pipeline
CI #1	11/08 13:05	1m 58s
CI #2	11/08 14:36	1m 29s
CI #3	13/08 13:20	1m 43s
CI #4	13/08 14:45	1m 37s
Moyenne	—	~1m 42s

Détail — Change Failure Rate (tentatives de déploiement)

Tentative	Cause de l'échec	Résultat
1 (après merge PR#3)	Tag GHCR en majuscules	Échec
2 (après merge PR#5)	Correctif partiel — tags encore en majuscules	Échec
3 (après merge PR#6)	gradlew: Permission denied (build Docker)	Échec
4 (après merge PR#7)	Correctif chmod +x appliqué	Succès

4.2 KPI opérationnels complémentaires

KPI	Valeur mesurée	Source
Temps de build back-end (tests)	1m10s - 1m51s (CI) / jusqu'à 62 min en build Docker complet	Gradle, CI
Temps de build front-end	~2 min (CI) / 24s (local)	npm / ng, CI
Couverture de code back-end	56.4 %	SonarQube Cloud
Couverture de code front-end	27.0 %	SonarQube Cloud
Vulnérabilités dépendances (npm)	130 (2 critiques, 62 hautes, 51 modérées, 15 faibles)	Dependabot
Quality Gate SonarQube (back)	Passed	SonarCloud
Répartition des logs par niveau	25 INFO / 5 WARN / 0 ERROR	Kibana — dashboard Projet-7

Le temps de build Docker back-end (jusqu'à 62 min) est fortement dépendant de la qualité de connexion réseau locale utilisée lors du développement, pas du code lui-même — sur les runners GitHub Actions (réseau stable), les mêmes builds s'exécutent en quelques minutes.

5. Analyse des métriques

Le Change Failure Rate de 75 % constaté sur la mise en place du déploiement continu s'explique par trois causes distinctes, toutes liées à des différences d'environnement invisibles en local : la casse imposée par GHCR sur les noms d'image (résolue par une conversion en minuscules), et la perte des droits d'exécution Unix lors d'un COPY Docker sur un runner Linux propre (résolue par un `chmod +x` explicite).

Chacun de ces échecs a été détecté et corrigé en moyenne en 15 minutes (MTTR), avant tout impact réel — ce qui démontre concrètement la valeur du pipeline CI/CD : transformer des problèmes qui seraient survenus en production en incidents bénins détectés en amont.

Le Lead Time du pipeline (~1m 42s) est jugé excellent et stable sur les 4 mesures réalisées. Il ne couvre toutefois que la portion outillée (commit → validé), pas le temps de développement humain, non mesurable techniquement.

Le croisement avec les données ELK confirme la cohérence de l'ensemble : sur la période observée, aucun log de niveau ERROR n'a été enregistré, ce qui explique l'absence de MTTR applicatif réel à mesurer — l'application n'a connu aucun incident de production, seulement des échecs de pipeline, catégorie différente et moins critique.

Dashboard Kibana associé (données ELK croisées)

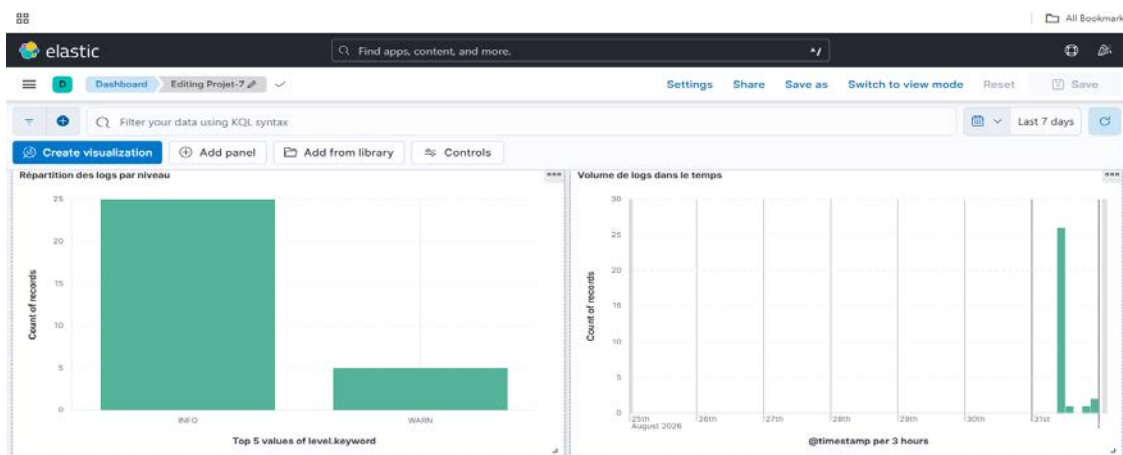


Figure 2 — Dashboard Kibana "Projet-7" : répartition des logs par niveau et volume dans le temps

Cohérence vérifiée : le taux de logs WARN observés (5 sur 30, ~17 %) correspond exactement au test volontaire réalisé (envoi d'un JSON malformé), et non à une instabilité réelle de l'application.

6. Plan de sécurité

6.1 Rôle de l'analyse SonarQube Cloud

SonarQube Cloud analyse le code à chaque exécution du pipeline (push et Pull Request), sur deux projets distincts (back-end et front-end). Il calcule une note de sécurité, fiabilité et maintenabilité, avec un Quality Gate bloquant avant toute fusion vers main.

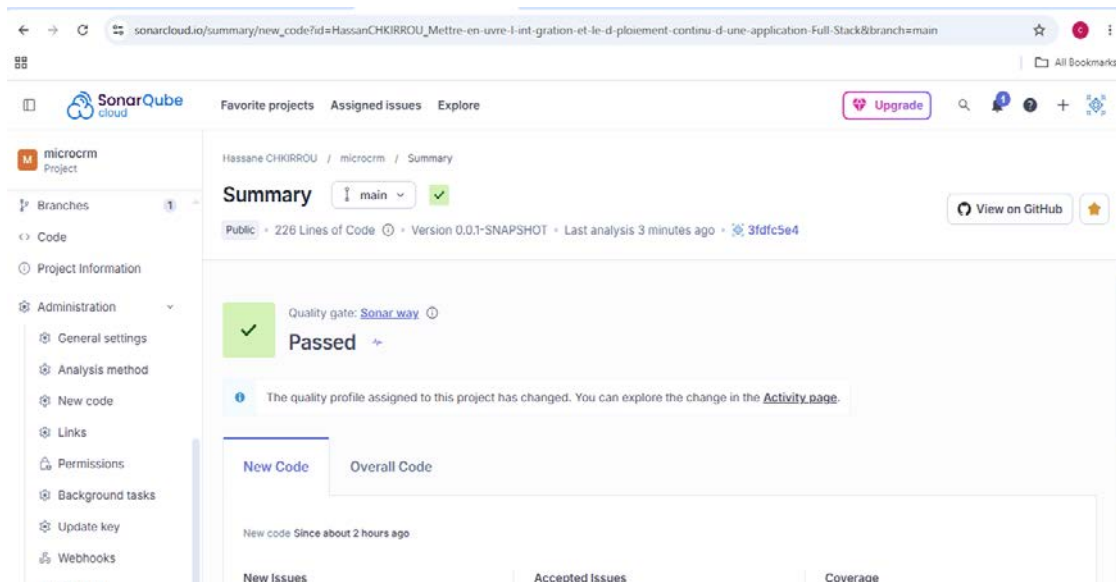


Figure 3 — Quality Gate "Passed" sur le projet back-end (SonarQube Cloud)

Vue d'ensemble consolidée (résultats détaillés)

Métrique	Back-end (Java)	Front-end (Angular)
Lignes de code	226	733
Bugs	2 (Rating A)	9 (Rating C)
Vulnerabilities	3 (Rating C)	1 (Rating B)
Security Hotspots	0 (Rating A)	0 (Rating A)
Code Smells	8 (Rating A)	37 (Rating A)
Duplication	9.2 %	0.0 %
Complexité cyclomatique	33	58
Complexité cognitive	3	15

6.2 Vulnérabilités identifiées (distinctes des code smells)

Point de vigilance : ne pas confondre une vulnérabilité (faille de sécurité réelle) avec un code smell (dette technique, sans impact sécuritaire).

#	Fichier	Règle	Sévérité	Catégorie OWASP
1-3	OrganizationRepository.java, PersonRepository.java, SpringDataRestCustomization.java	CORS activé sans restriction explicite	Medium	A05:2021
4	front/src/index.html	Attribut Subresource Integrity manquant	Low	A08:2021

4 vulnérabilités réelles (type Security) strictement séparées des 45 code smells (type Maintainability) et des 27 problèmes de fiabilité (type Reliability, détaillés ci-dessous).

Problèmes de fiabilité identifiés (non sécuritaires)

- 13 occurrences de « Unexpected await of a non-Promise value » (organization.service.ts, person.service.ts) — sévérité High. Bug potentiel : await utilisé sur un Observable RxJS.
- 14 occurrences d'accessibilité WCAG (labels non associés, attributs id manquants) — sévérité Medium. Risque de conformité, pas de sécurité.

6.3 Bonnes pratiques de gestion des secrets et dépendances

- Aucun secret en clair dans le code ou les workflows : SONAR_TOKEN et GITHUB_TOKEN gérés via GitHub Secrets
- Dependabot actif nativement, surveille les dépendances npm et ouvre des Pull Requests de mise à jour automatiquement
- Les workflows déclenchés par des PR de forks externes n'ont pas accès aux secrets (comportement par défaut GitHub, non modifié)

6.4 Solutions d'amélioration proposées

Problème	Solution concrète
CORS Spring Data REST	Configurer explicitement @CrossOrigin(origins="...") sur chaque repository
SRI manquant	Ajouter integrity + crossorigin="anonymous" sur les ressources chargées depuis un CDN
await sur Observable	Remplacer par firstValueFrom(...) (RxJS 7+) ou refactorer en .subscribe()
Accessibilité WCAG	Associer chaque label à son input via for/id — correctif mécanique, faible effort
Dépendances Gradle non surveillées	Étendre Dependabot au back-end (actuellement npm uniquement)

7. Plan de sauvegarde

7.1 Éléments sauvegardés

Élément	Méthode	Fréquence
Code source	Versionné sur GitHub (branches main / dev)	Continue (à chaque commit)
Images Docker publiées	Conservées sur GHCR, taguées par SHA de commit	Continue, conservation indéfinie
Configuration CI/CD	Versionnée avec le code (.github/workflows/)	Continue
Secrets	GitHub Secrets, répliqués par l'infrastructure GitHub	Géré par la plateforme
Base de données applicative	Sans objet actuellement	—

Point de vigilance majeur : la base de données HSQLDB tourne en mémoire. Aucune donnée créée par les utilisateurs n'est persistée d'un redémarrage à l'autre ; seul le jeu de données de démonstration est rechargé automatiquement. Il n'y a donc, en l'état, aucune donnée applicative à sauvegarder.

7.2 Recommandation pour une évolution future

Si une base de données persistante (ex. PostgreSQL en volume Docker) était introduite, il faudrait alors prévoir des exports réguliers (ex. pg_dump quotidien) avec une politique de rétention à définir selon la criticité des données.

8. Plan de mise à jour

8.1 Mise à jour du code applicatif

Toute évolution suit le flux standard décrit dans le plan de déploiement (section 2) : développement sur dev, validation par le pipeline CI, Pull Request vers main, publication automatique.

8.2 Gestion des dépendances

Type	Méthode actuelle
Dépendances npm (front)	Dependabot actif — Pull Requests automatiques
Dépendances Gradle (back)	Vérification manuelle (./gradlew dependencies) — amélioration possible
Images Docker de base	Vérification périodique manuelle via Docker Scout

8.3 Bonnes pratiques retenues

- Toujours passer par une Pull Request dev → main, jamais de commit direct sur main
- Vérifier systématiquement le Quality Gate SonarQube avant tout merge
- Revoir périodiquement les recommandations Docker Scout sur les images de base
- Traiter les Pull Requests Dependabot au fil de l'eau plutôt que de les laisser s'accumuler

Justification : ces pratiques découlent directement du Change Failure Rate de 75 % mesuré en section 4 — l'expérience du projet montre que des problèmes invisibles en local (permissions, casse de registre) ne se révèlent qu'en environnement CI propre, d'où l'importance de ne jamais court-circuiter le pipeline.

9. Synthèse des recommandations d'amélioration continue

Récapitulatif de toutes les recommandations détaillées dans les sections précédentes, chacune justifiée par une observation concrète du projet.

Recommandation	Référence
Corriger les 4 vulnérabilités CORS/SRI identifiées	Section 6.2 — corrections rapides, faible effort, gain de sécurité réel
Remplacer les await sur Observable par firstValueFrom	Section 6.2 — 13 occurrences, sévérité High, risque de comportement asynchrone incorrect
Augmenter la couverture de tests front-end (27 % → 50 %+)	Section 4.2 — couverture la plus faible du projet
Ajouter Dependabot pour Gradle (pas seulement npm)	Sections 6.4 / 8.2 — 130 vulnérabilités surveillées côté npm uniquement
Provoquer et suivre de vraies erreurs applicatives (niveau ERROR)	Section 5 — 0 log ERROR observé, limite la pertinence du MTTR applicatif
Étendre le monitoring ELK au front-end	Section 5 — nécessiterait un point d'entrée serveur dédié
Réduire le Change Failure Rate en fiabilisant tests locaux vs CI	Section 4.1 — 75 % d'échecs dus à des différences d'environnement non détectables en local
Corriger les 14 problèmes d'accessibilité WCAG	Section 6.2 — correctifs mécaniques à faible effort
Introduire une base de données persistante	Section 7 — HSQLDB en mémoire, limite la portée du plan de sauvegarde